Computer Architecture (Practical Class)
Assembly: Stack and Functions - Parameters and Local Variables

Luís Nogueira Nuno Pereira Raquel Faria

Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

{lmn,nap,arf}@isep.ipp.pt

2018/2019

Functions - Defining input values

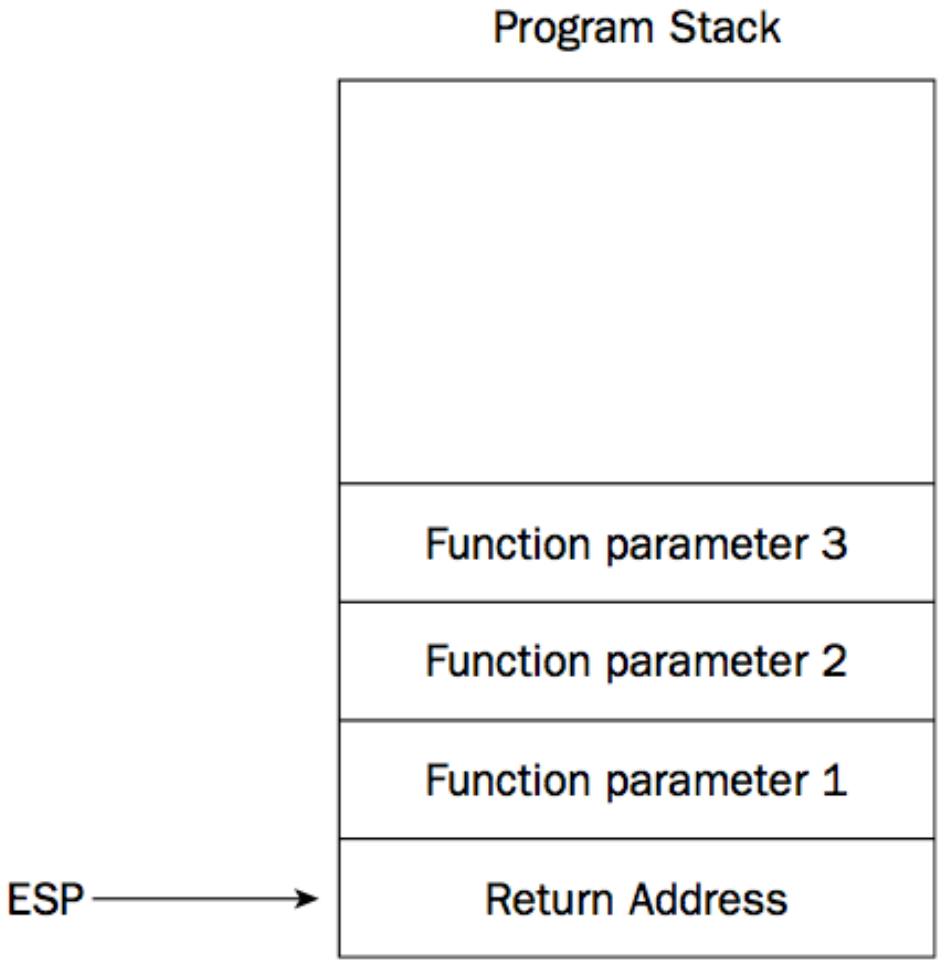
- Many (if not most) functions require some sort of input data
- You must decide how your program will pass this information along to the function
  - Using **registers**, and/or **global variables** and/or **the stack**
  - Trying to keep track of which function uses which registers and global variables, or which registers and global variables are used to pass which parameters, can be a nightmare
- The C solution for passing input values to functions is to use the stack
  - This method works equally as well for any Assembly program, even if it wasn't derived from a C program

Passing parameters on the stack (1/2)

- Before a function call is made, the main program places the required input parameters for the function on the top of the stack
- Of course, the programmer must know the order in which the function is expecting the data values to be placed on the stack
- The C style requires placing parameters on the stack in reverse order from the prototype for the function
- When the `call` instruction is executed, it places the return address from the calling program onto the top of the stack as well, so the function knows where to return

Passing parameters on the stack (2/2)

- All these details leave the stack in the state shown in this picture (assume a function with three input parameters)

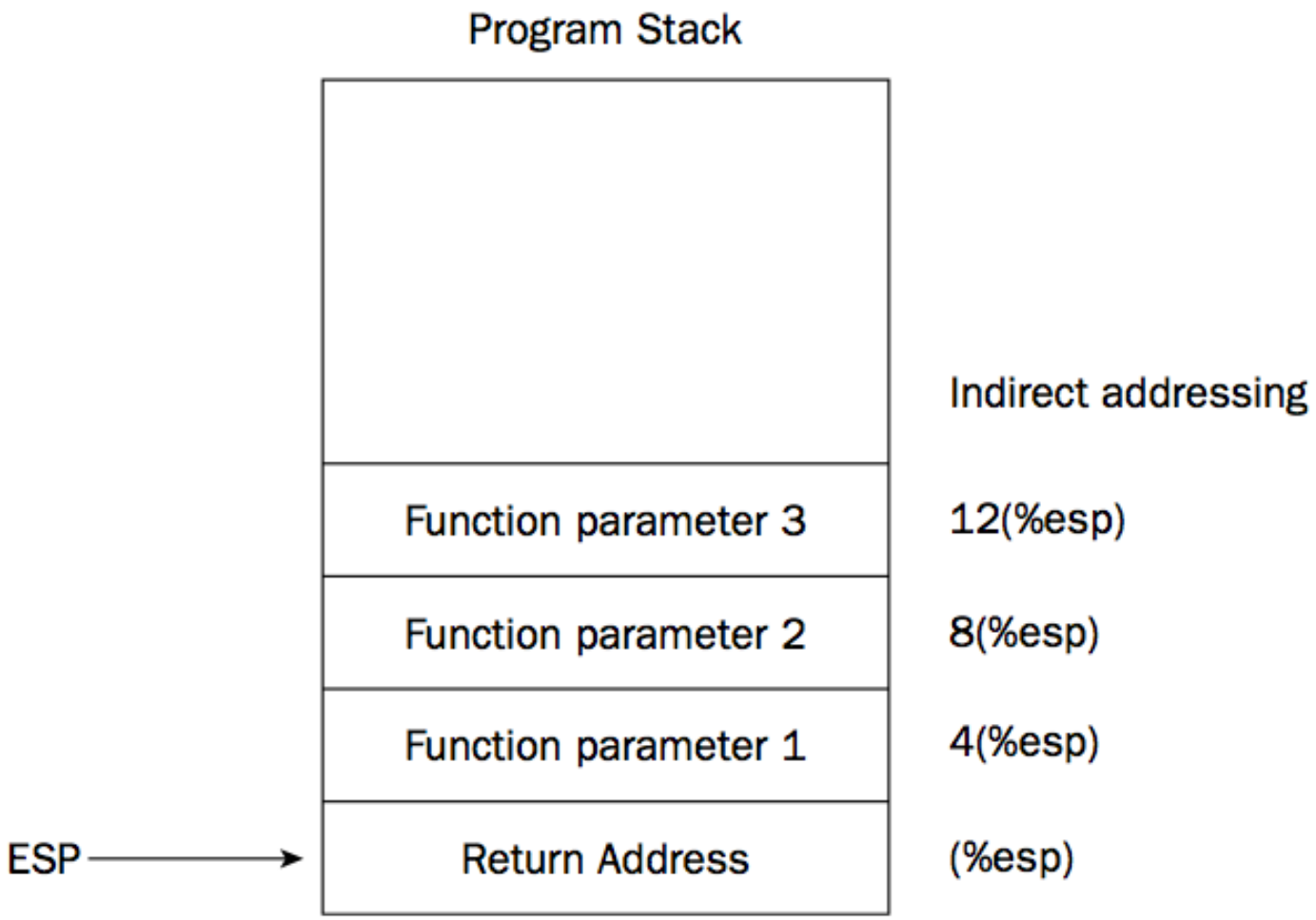


Accessing parameters (1/2)

- The stack pointer (ESP) points to the top of the stack, where the return address is located
  - All of the input parameters for the function are located "underneath" the return address on the stack
- Popping values off of the stack to retrieve the input parameters would cause a problem, as the return address might be lost in the process
- Each parameter can be indirectly accessed via the offset from the ESP register, without having to POP values off of the stack
  - `movl (%esp), %eax` - copies (without popping!) the content on the top of the stack to the EAX register
  - `movl 4(%esp), %eax` - which value is copied in this case?

Accessing parameters (2/2)

- Using indirect addressing with the ESP register to access the input parameters

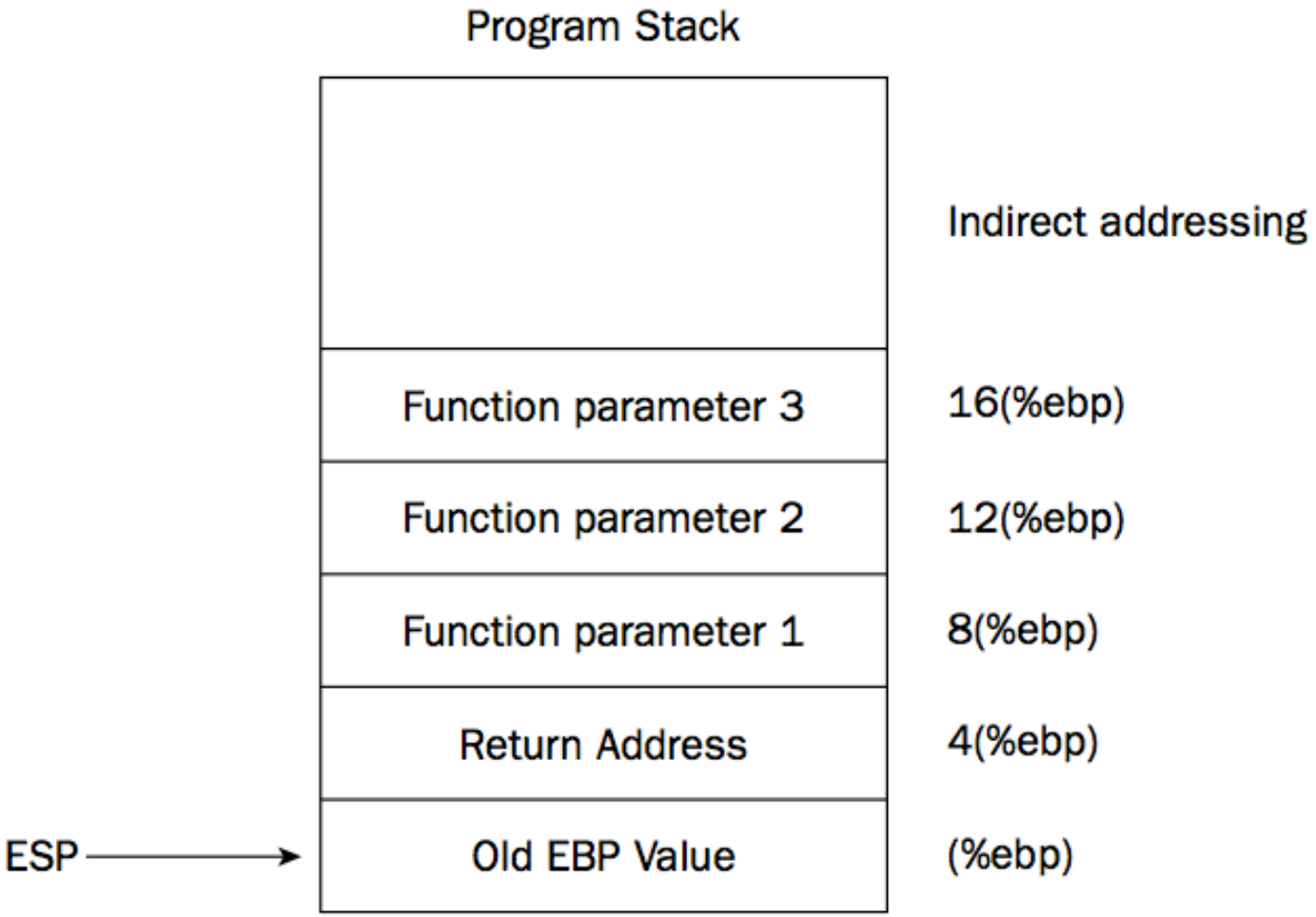


Using EBP as the base pointer (1/2)

- There is a problem with this technique! What happens if data is pushed to the stack?
- If this happens, it would change the location of the ESP stack pointer and throw off the indirect addressing values for accessing the parameters in the stack
- To avoid this problem, it is common practice to copy the ESP register value to the EBP register when entering the function
- This ensures that there is a register that always contains the correct pointer to the top of the stack when the function is called (a `push` does not changes EBP)
- To avoid corrupting the original EBP register if it is used in the main program, before the ESP register value is copied, the EBP register value is also placed on the stack

Using EBP as the base pointer (2/2)

- All these details leave the stack in the state shown in this picture (assume a function with three input parameters)



Prologue and Epilogue (1/2)

- The technique has created a standard set of instructions that are found in all functions written using the C function style technique

Function Template (http://codepad.org/9Zym1BEP)

```
function:
   # prologue
   pushl %ebp        # save the original value of EBP
   movl %esp,%ebp    # copy the current stack pointer to EBP

   # callee is responsible for:
   # %ebx, %esi and %edi
   # save only those that are used
   ...

   # function body
   ...

   # restore callee saved registers
   ...

   #epilogue
   movl %ebp,%esp    # retrieve the original ESP value
   popl %ebp         # restore the original EBP value
   ret
```

Prologue and Epilogue (2/2)

- The prologue saves the original value of EBP to the top of the stack, and then copy the current ESP stack pointer (now pointing to the original value of EBP in the stack) to the EBP register
- The epilogue retrieves the original value in the ESP register that was stored in the EBP register, and restore the original EBP register value.
- This ensures that any data placed on the stack within the function but not cleaned off will be discarded when execution returns to the main program (otherwise, the RET instruction could return to the wrong memory location)

Cleaning out the stack (1/2)

- Before the function is called, the calling program places all of the required input values onto the stack
- When the function returns, those values are still on the stack (since the function accessed them without popping them off of the stack)
- If the main program uses the stack for other things, most likely it will want to remove the old input values from the stack to get the stack back to where it was before the function call
- While you can use the POP instruction to do this, you can also just move the ESP stack pointer back to the original location before the function call

Cleaning out the stack (2/2)

- Adding back the size of the data elements pushed onto the stack using the ADD instruction does this

Cleaning the Stack after Return

```
function:
  ...

  pushl %eax    # push 2nd parameter on stack
  pushl %ebx    # push 1st parameter on stack
  call soma
  addl $8,%esp # get the stack back to where it was before the function call

  ...
```

Example - Passing parameters

Passing Parameters

```
.global main_program

main_program:
  #prologue
  pushl %ebp
  movl %esp, %ebp

  pushl $3       # 2nd parameter, expoenent
  pushl $2       # 1st parameter, base
  call power
  addl $8, %esp # clean parameters from stack

  # epilogue
  movl %ebp, %esp
  popl %ebp
  ret
```

Example - Accessing parameters

Accessing Parameters

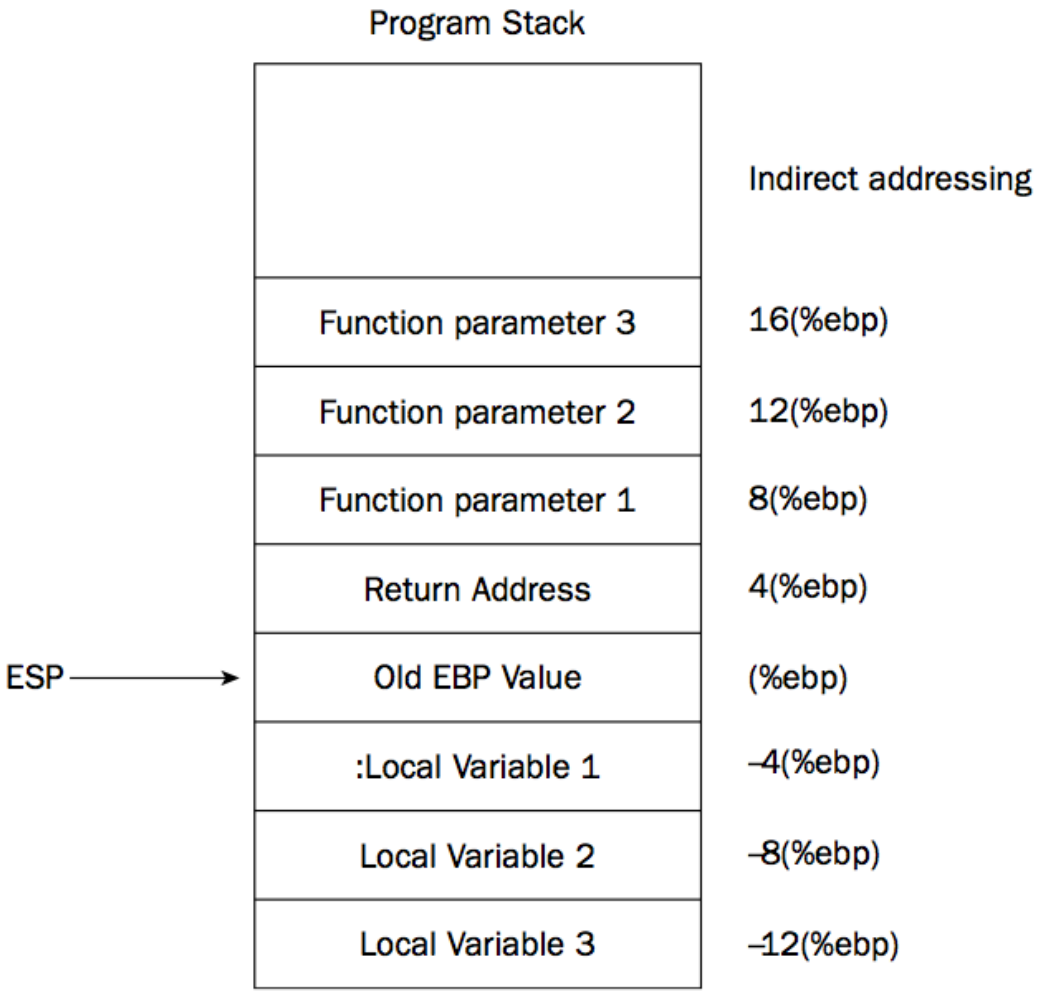
```
power:
  pushl %ebp              # prologue
  movl %esp, %ebp

  pushl %ebx              # callee saves %ebx before using it

  movl 8(%ebp), %ebx      # base on %ebx
  movl 12(%ebp), %ecx     # exponent on %ecx
  movl %ebx, %eax         # initial result in %eax

power_loop_start:
  cmpl $1, %ecx           # if exponenent is 1, go to end_power
  je end_power
  imull %ebx, %eax        # multiplies current result by base
  decl %ecx               # decreases the exponent
  jmp power_loop_start    # jumps to next expoenent

end_power:
  popl %ebx               # calle restores %ebx before ret

  movl %ebp, %esp         # epilogue
  popl %ebp
  ret
```

Reserving space for local variables (1/6)

- When looking for an easy location for working storage for data elements within a function, the stack again comes to the rescue
- The idea is to allocate storage for these variables on entry to the function, and release the storage on return
- This space can be reserved after setting the EBP register to point to the top of the stack without affecting how the function's parameters are accessed
- Assuming 4-byte data values, the first local variable would be accessed by referencing `-4(%ebp)`, while the second local variable would be accessed by referencing `-8(%ebp)`, ...

Reserving space for local variables (2/6)

- Using indirect addressing with EBP to access input parameters and local variables



Reserving space for local variables (3/6)

- There is still one lingering problem with this setup! What happens if the function pushes any data onto the stack?
- ESP still points to the location before the local variables were placed
- Any `push` will overwrite those variables
- To solve this problem, another line is added to the prologue to reserve a set amount of stack space for local variables by subtracting the value from the ESP register

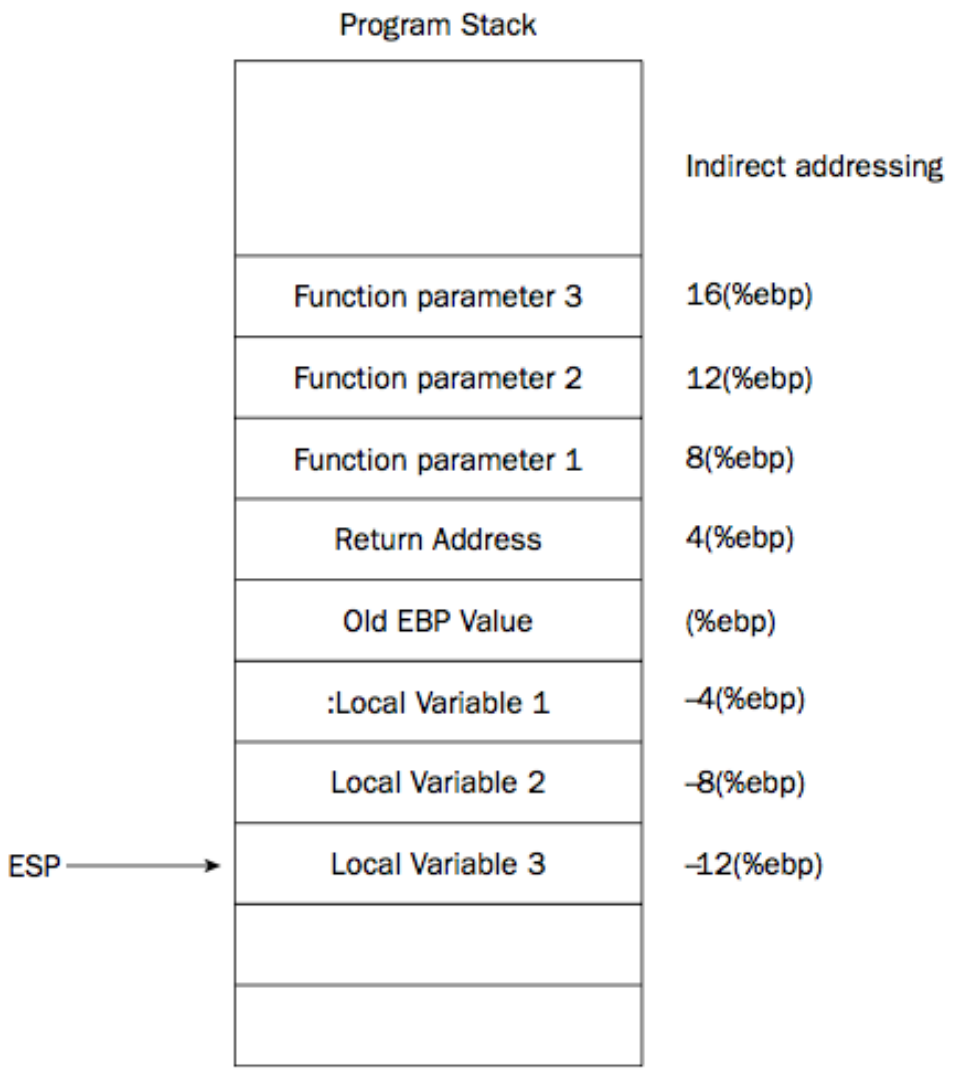
Reserving space for local variables (4/6)

- You must remember to reserve enough space for all of the local variables needed in the function

Reserving Space for Local Variable

```
function:
    # prologue
    pushl %ebp
    movl %esp, %ebp
    subl $12, %esp    # reserve 12 bytes for local variables

    ...
```

Reserving space for local variables (5/6)



Reserving space for local variables (6/6)

- Now, if any data is pushed onto the stack, it will be placed beneath the local variables
- Local variables are then preserved and they can still be accessed via EBP
- When the end of the function is reached and the ESP register is set back to its original value, the local variables will be lost from the stack
- They will not be directly accessible using the ESP or EBP registers from the calling function (thus the term "local variables")

Example - Using a local variable

Using a Local Variable for Temporary Storage

```
power:
  pushl %ebp                # prologue
  movl %esp, %ebp
  subl $4,%esp              # reserves space for a local variable

  pushl %ebx                # callee saves %ebx before using it
  movl 8(%ebp), %ebx        # base on %ebx
  movl 12(%ebp), %ecx       # exponent on %ecx
  movl %ebx, -4(%ebp)       # initial result in local variable

power_loop_start:
  cmpl $1, %ecx             # if exponenent is 1, go to end_power
  je end_power
  imull %ebx,-4(%ebp)       # multiplies current result by base
  decl %ecx                 # decreases the exponent
  jmp power_loop_start      # jumps to next expoenent

end_power:
  movl -4(%ebp), %eax       # final result on %eax
  popl %ebx                 # calle restores %ebx before ret
  movl %ebp, %esp           # epilogue
  popl %ebp
  ret
```